New York University
Computer Science Department
Courant Institute of Mathematical Sciences

Course Title: Data Communication & Networks
Course Number: CSCI-GA.2662-001
Instructor: Jean-Claude Franchitti
Session: 3

Lab #3 – Application Layer

I. Due

Monday February 23, 2026, at the beginning of class.

II. Objectives

1. Understand the basic principles of computer networks and related protocols
2. Understand how to analyze application layer protocols
3. Develop a simple network application, explore and understand DNS protocol.

III. References

1. Slides and handouts posted on the course Web site
2. Textbook chapters as applicable

IV. Software Required

1. Microsoft Word.
2. Win Zip as necessary.
3. Wireshark,
4. Docker.
5. IBM Cloud
6. Kubernetes (K8s).

V. Lab Instructions

1. Problem 1 – Protocols Analysis:

Having gotten our feet wet with the Wireshark packet sniffer in the introductory lab, we are now ready to use Wireshark to investigate protocols in operation. In this lab, we will explore several aspects of the HTTP protocol: the basic GET/response interaction, HTTP message formats, retrieving large HTML files, retrieving HTML files with embedded objects, and HTTP authentication and security. Before beginning these labs, you might want to review Section 2.2 of the text.

You might also want to review the introductory Wireshark Lab (session 1 lab), where we noted that before you run Wireshark, there are several things you should check in your network connection and browser configuration:

- Make sure you are not running a VPN (virtual private network) service. We will learn about VPNs later in this course. But for now, all you need to know is that when you are running a VPN service, upper-layer protocol information (HTTP, TCP) that is sent by your computer may be encrypted. This makes it impossible to use Wireshark to look at what is going on inside these protocols!
- Make sure your browser program is not, by default, using the HTTP/3 protocol or the QUIC protocol. We will learn about HTTP/3 and QUIC in the course. But for now, all you need to know is that when you are running HTTP/3 or QUIC, upper-layer protocol information (HTTP, TCP) sent by your computer will be encrypted. As of 2025, most popular browsers have adopted HTTP/3 and QUIC as defaults. A document that describes how to disable HTTP/3 AND QUIC as the defaults for your browser is here: https://techysnoop.com/disable-quic-protocol-in-chrome-edge-firefox/.
- Make sure then when you go to a URL, that the URL starts with "http://" rather than with https:// as the latter case will encrypt frames.
- Make sure privacy and browser privacy settings are turned off.
- Make sure you clear your browser cache and browser history before you start.

a. The Basic HTTP GET/response interaction

Let us begin our exploration of HTTP by downloading a very simple HTML file - one that is very short and contains no embedded objects. Do the following:

1. Start up your web browser.
2. Start up the Wireshark packet sniffer, as described in the Introductory lab (but do not yet begin packet capture). Enter "http" (just the letters, not the quotation marks) in the display-filter-specification window, so that only captured HTTP messages will be displayed later in the packet-listing window. (We are only interested in the HTTP protocol here, and do not want to see the clutter of all captured packets).
3. Wait a bit more than one minute (we will see why shortly), and then begin Wireshark packet capture.
4. Enter the following to your browser:
   http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file1.html
   Your browser should display the very simple, one-line HTML file.
5. Stop Wireshark packet capture.

Your Wireshark window should look similar to the window shown in Figure 1. If you are unable to run Wireshark on a live network connection, you can download a packet trace that was created when the steps above were followed.[1]

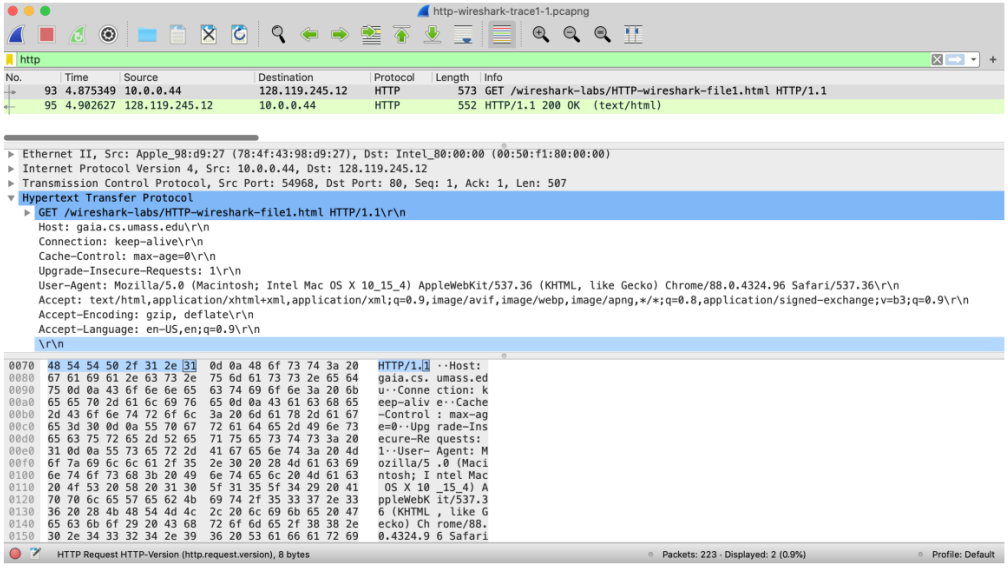


Figure 1: Wireshark Display after http://gaia.cs.umass.edu/wireshark-labs/ HTTP- wireshark-file1.html has been retrieved by your browser

The example in Figure 1 shows in the packet-listing window that two HTTP messages were captured: the GET message (from your browser to the gaia.cs.umass.edu web server) and the response message from the server to your browser. The packet-contents window shows details of the selected message (in this case the HTTP OK message, which is highlighted in the packet-listing window). Recall that since the HTTP message was carried inside a TCP segment, which was carried inside an IP datagram, which was carried within an Ethernet frame, Wireshark displays the Frame, Ethernet, IP, and TCP packet information as well. We want to minimize the amount of non-HTTP data displayed (we are interested in HTTP here, and will be investigating these other protocols is later labs), so make sure the boxes at the far left of the Frame, Ethernet, IP and TCP information have a plus sign or a right-pointing triangle (which means there is hidden, un-displayed information), and the HTTP line

[1] Download the zip file https://www-net.cs.umass.edu/wireshark-labs/wireshark-traces-9e.zip and extract the file *http-wireshark-trace1-1*. This trace file can be used to answer these Wireshark lab questions without actually capturing packets on your own. The traces in this zip file were collected by Wireshark running on the computer of one of the author's of the textbook for the course, while performing the steps indicated above. Once you have downloaded the trace, you can load it into Wireshark and view the trace using the *File* pull down menu, choosing *Open*, and then selecting the *http-ethereal-trace-1-1* trace file. The resulting display should look similar to Figure 1. (The Wireshark user interface displays just a bit differently on different operating systems, and in different versions of Wireshark).

has a minus sign or a down-pointing triangle (which means that all information about the HTTP message is displayed).

Note: You should ignore any HTTP GET and response for favicon.ico. If you see a reference to this file, it is your browser automatically asking the server if it (the server) has a small icon file that should be displayed next to the displayed URL in your browser. We will ignore references to this pesky file in this lab.

By looking at the information in the HTTP GET and response messages, answer the following questions. When answering the following questions, you should print out the GET and response messages (see the introductory Wireshark lab for an explanation of how to do this) and indicate where in the message you have found the information that answers the following questions. When you hand in your assignment, annotate the output so that it is clear where in the output you are getting the information for your answer (e.g., for our classes, we ask that students markup paper copies with a pen, or annotate electronic copies with text in a colored font).

1. Is your browser running HTTP version 1.0, 1.1, or 2? What version of HTTP is the server running?
2. What languages (if any) does your browser indicate that it can accept to the server?
3. What is the IP address of your computer? What is the address of the gaia.cs.umass.edu server?
4. What is the status code returned from the server to your browser?
5. When was the HTML file that you are retrieving last modified at the server?
6. How many bytes of content are being returned to your browser?
7. By inspecting the raw data in the packet content window, do you see any headers within the data that are not displayed in the packet-listing window? If so, name one.

In your answer to question 5 above (assuming you are running Wireshark "live", as opposed to using a trace file recorded earlier), you might have been surprised to find that the document you just retrieved was last modified within a minute before you downloaded the document. That is because (for this particular file), the gaia.cs.umass.edu server is setting the file's last-modified time to be the current time and is doing so once per minute. Thus, if you wait a minute between accesses, the file will appear to have been recently modified, and hence your browser will download a "new" copy of the document.

b. The HTTP conditional GET/response interaction

Recall from Section 2.2.5 of the textbook, that most web browsers perform object caching and thus perform a conditional GET when retrieving an HTTP object. Before performing the steps below, make sure your browser's cache is empty. See https://www.howtogeek.com/304218/how-to-clear-your-history-in-any-browser/ for instructions on clearing your browser cache. Now do the following:

- Start up your web browser, and make sure your browser's cache is cleared, as discussed above.
- Start up the Wireshark packet sniffer
- Enter the following URL into your browser http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file2.html Your browser should display a very simple five-line HTML file.
- Quickly enter the same URL into your browser again (or simply select the refresh button on your browser)
- Stop Wireshark packet capture and enter "http" (again in lower case without the quotation marks) in the display-filter-specification window, so that only captured HTTP messages will be displayed later in the packet-listing window.

  Note: If you are unable to run Wireshark on a live network connection (or unable to get your browser to issue an If-Modified-Since field on the second HTTP GET request), you can download the zip file https://www-net.cs.umass.edu/wireshark-labs/wireshark-traces-9e.zip and extract the trace *file http-wireshark-trace2-1* and use it to answer the questions below; see footnote 1. This trace file was gathered while performing the steps above on the computers used by one of the authors of the book for the course.

  Hint: ideally, you should see an If-Modified-Since header since you have just downloaded this page a few seconds ago. However, depending on the browser you are using, and the format of the server's earlier response to your initial GET, your browser may not include an If-Modified-Since even if the document has been downloaded and caches. The Chrome browser is pretty good at regularly using If-Modified-Since. But Safari and Firefox are much more finicky about when to use If-Modified-Since. Life is not always as easy in practice as it is in theory!

Answer the following questions:

1. Inspect the contents of the first HTTP GET request from your browser to the server. Do you see an "IF-MODIFIED-SINCE" line in the HTTP GET?
2. Inspect the contents of the server response. Did the server explicitly return the contents of the file? How can you tell?
3. Now inspect the contents of the second HTTP GET request from your browser to the server. Do you see an "IF-MODIFIED-SINCE:" line in the HTTP GET? If so, what information follows the "IF-MODIFIED-SINCE:" header?
4. What is the HTTP status code and phrase returned from the server in response to this second HTTP GET? Did the server explicitly return the contents of the file? Please explain.

c. Retrieving long documents

In our examples thus far, the documents retrieved have been simple and short HTML files. Let us next see what happens when we download a long HTML file. Do the following:

- Start up your web browser, and make sure your browser's cache is cleared, as discussed above.
- Start up the Wireshark packet sniffer
- Enter the following URL into your browser http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file3.html Your browser should display the rather lengthy US Bill of Rights.
- Stop Wireshark packet capture and enter "http" in the display-filter-specification window, so that only captured HTTP messages will be displayed.

  Note: If you are unable to run Wireshark on a live network connection, you can download the zip file https://www-net.cs.umass.edu/wireshark-labs/wireshark-traces-9e.zip and extract the trace file *http-wireshark-trace3-1*. You can then use that packet trace to answer the questions below. This trace file was gathered while performing the steps above on the computer of one of the author of the textbook for the course.)

In the packet-listing window, you should see your HTTP GET message, followed by a multiple-packet TCP response to your HTTP GET request. Make sure that your Wireshark display filter is cleared so that the multi-packet TCP response is displayed in the packet listing.

This multiple-packet response deserves a bit of explanation. Recall from Section 2.2 (see Figure 2.9 in the textbook) that the HTTP response message consists of a status line, followed by header lines, followed by a blank line, followed by the entity body. In the case of our HTTP GET, the entity body in the response is the *entire* requested HTML file. In our case here, the HTML file is rather long, and at 4500 bytes is too large to fit into one TCP packet. The single HTTP response message is thus broken into several pieces by TCP, with each piece being contained within a separate TCP segment (see Figure 1.24 in the text). In recent versions of Wireshark, Wireshark indicates each TCP segment as a separate packet, and the fact that the single HTTP response was fragmented across multiple TCP packets is indicated by the "TCP segment of a reassembled PDU" in the Info column of the Wireshark display.

Note: Earlier versions of Wireshark used the "Continuation" phrase to indicated that the entire content of an HTTP message was broken across multiple TCP segments.. We stress here that there is no "Continuation" message in HTTP! In some versions of Wireshark, instead of separate packets for each segment, Wireshark shows one packet. If you look at the part above Hypertext Transfer Protocol, you should see a section titled: "… Reassembled TCP Segments…".

So, please answer the questions according to the version of Wireshark you are using.

Answer the following questions:

1. How many HTTP GET request messages did your browser send? Which packet number in the trace contains the GET message for the Bill or Rights?
2. Which packet number in the trace contains the status code and phrase associated with the response to the HTTP GET request?
3. What is the status code and phrase in the response?
4. How many data-containing TCP segments were needed to carry the single HTTP response and the text of the Bill of Rights?.

d. HTML documents with embedded objects

Now that we have seen how Wireshark displays the captured packet traffic for large HTML files, we can look at what happens when your browser downloads a file with embedded objects, i.e., a file that includes other objects (in the example below, image files) that are stored on another server(s).

Do the following:

- Start up your web browser, and make sure your browser's cache is cleared, as discussed above.
- Start up the Wireshark packet sniffer
- Enter the following URL into your browser:
  http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file4.html

  Your browser should display a short HTML file with two images. These two images are referenced in the base HTML file. That is, the images themselves are not contained in the HTML; instead the URLs for the images are contained in the downloaded HTML file. As discussed in the textbook, your browser will have to retrieve these logos from the indicated web sites. The textbook publisher's logo is retrieved from the gaia.cs.umass.edu web site. The image of the cover for the 8th edition is actually stored on a server located in France.
- Stop Wireshark packet capture and enter "http" in the display-filter-specification window, so that only captured HTTP messages are displayed.

  Note: If you are unable to run Wireshark on a live network connection, you can download the zip file https://www-net.cs.umass.edu/wireshark-labs/wireshark-traces-9e.zip and extract the trace file *http-wireshark-trace4-1*, and then use that packet trace to answer the questions below. This trace file was gathered while performing the steps above on one the computer of one of the authors of the textbook for the course.)

Answer the following questions:

1. How many HTTP GET request messages did your browser send? To which Internet addresses were these GET requests sent?
2. Can you tell whether your browser downloaded the two images serially, or whether they were downloaded from the two web sites in parallel? Please explain.

e. HTTP authentication

Finally, let us try visiting a web site that is password-protected and examine the sequence of HTTP message exchanged for such a site. The URL http://gaia.cs.umass.edu/wireshark-labs/protected_pages/HTTP-wireshark-file5.html is password protected. The username is "wireshark-students" (without the quotes), and the password is "network" (again, without the quotes). So let us access this "secure" password-protected site. Do the following:

- Make sure your browser's cache is cleared, as discussed above, and close down your browser. Then, start up your browser
- Start up the Wireshark packet sniffer
- Enter the following URL into your browser:
  http://gaia.cs.umass.edu/wireshark-labs/protected_pages/HTTP-wireshark-file5.html
  Type the requested user name and password into the pop up box.
- Stop Wireshark packet capture and enter "http" in the display-filter-specification window, so that only captured HTTP messages are displayed later in the packet-listing window.

  Note: If you are unable to run Wireshark on a live network connection, you can download the zip file https://www-net.cs.umass.edu/wireshark-labs/wireshark-traces-9e.zip and extract the trace file *http-wireshark-trace5-1*, and then use that trace file to answer the questions below. This trace file was gathered while performing the steps above on the computer of one of the authors of the textbook for the course.)

Now let us examine the Wireshark output. You might want to first read up on HTTP authentication by reviewing the easy-to-read material on "HTTP Access Authentication Framework" at:

http://frontier.userland.com/stories/storyReader$2159

Answer the following questions:

1. What is the server's response (status code and phrase) in response to the initial HTTP GET message from your browser?
2. When your browser sends the HTTP GET message for the second time,

what new field is included in the HTTP GET message?

The username (wireshark-students) and password (network) that you entered are encoded in the string of characters:

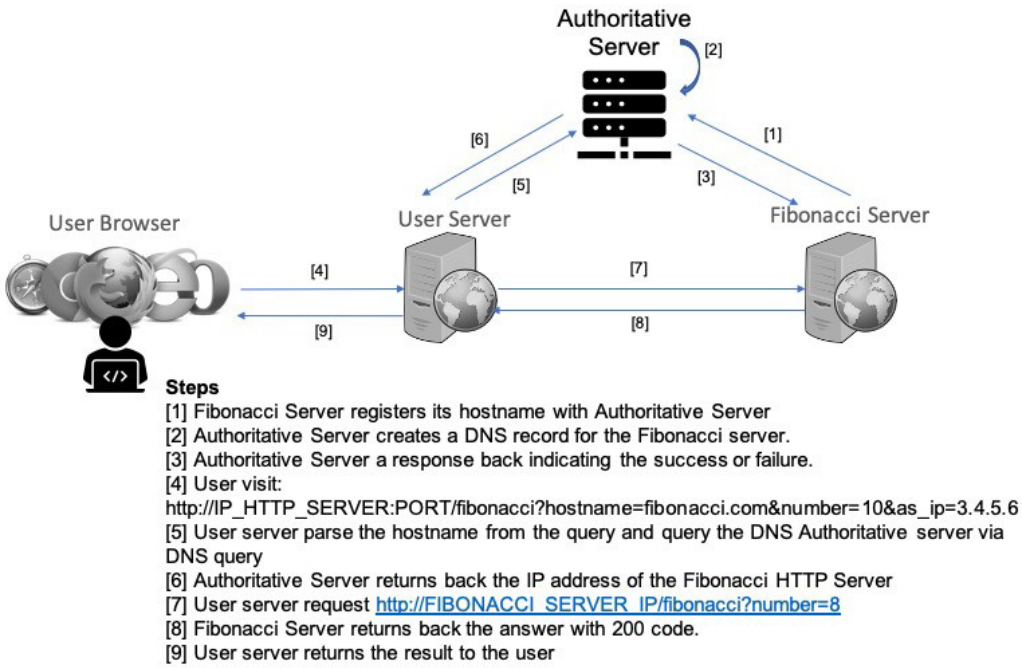
(d2lyZXNoYXJrLXN0dWRlbnRzOm5ldHdvcms=)

that follows the "`Authorization: Basic`" header in the client's HTTP GET message. While it may appear that your username and password are encrypted, they are simply encoded in a format known as Base64 format. The username and password are *not* encrypted! To see this, go to http://www.motobit.com/util/base64-decoder-encoder.asp and enter the base64-encoded string d2lyZXNoYXJrLXN0dWRlbnRz and decode. *Voila!* You have translated from Base64 encoding to ASCII encoding, and thus should see your username! To view the password, enter the remainder of the string Om5ldHdvcms= and press decode. Since anyone can download a tool like Wireshark and sniff packets (not just their own) passing by their network adaptor, and anyone can translate from Base64 to ASCII (you just did it!), it should be clear to you that simple passwords on WWW sites are not secure unless additional measures are taken.

Fear not! As we will see in Chapter 8 of the textbook, there are ways to make WWW access more secure. However, we will clearly need something that goes beyond the basic HTTP authentication framework!

2. **Problem 2 – Simplified authoritative server for a network of applications:**

Figure below shows an overview of the system that you need to develop.



The system that you will implement has 3 components:

a. **User server (US)** is a simple HTTP web server (e.g. Flask), running in port **8080**, that accepts a GET HTTP requests in path:

   "/fibonacci?hostname=fibonacci.com&fs_port=K&number=X&as_ip=Y &as_port= Z"

   The path accepts five parameters: **hostname and fs_port** which contains the hostname and port number of the server which will be queried to get a response for Fibonacci number for a given **sequence number X**. And **as_ip, as_port** which are the IP address and port number of the Authoritative Server (AS). If any of the parameters are missing, the server should return HTTP code **400** indicating a bad request. If the request is successful, it should return HTTP code **200** with the Fibonacci number for the sequence number X. the only problem is that US does not know the IP address of the given hostname and therefore needs to query its authoritative DNS server to learn about it.

b. **Fibonacci Server (FS)** is an HTTP web server, running in port **9090**, that provides the Fibonacci value for a given sequence number X. In order to achieve this objective, FS performs the following:

   1. **Hostname Specification**: FS accepts a HTTP PUT request at path "/**register**" where the body contains the name and IP address of the server (FS) and IP address of the Authoritative Server (AS). You can

choose any name of your choice, let us say "fibonacci.com". The body should contain **a json object** as below. FS should parse the body and noted that hostname and IP and IP, port of the AS and moved to step 2 for registration to authoritative server.

```
{
    "hostname":
    "fibonacci.com",
    "ip": "172.18.0.2",
    "as_ip": "10.9.10.2",
    "as_port": "30001"
}
```

2. **Registration to Authoritative Server** Once the request is retrieved at path "/register", the hostname needs be registered with Authoritative server (AS) via UDP on port **53533**. Your DNS message should include (Name, Value, Type, TTL) where the type in this case is "A" and TTL in **seconds**. Below is a simplified DNS registration request (please follow the format as below -- without the dashes--, each line ends with a new line):

   TYPE=A
   NAME=fibonacci.com VALUE=IP_ADDRESS TTL=10
   ----

3. Once registration is successful, returns back a HTTP response with code **201**.

4. Serve a path in "**/fibonacci?number=X**" which accepts HTTP GET request and returns Fibonacci number for the sequence number X. Return **200** as a status code if successful. If X is not an integer (for example a string) return **400** indicating the bad format.

c. Authoritative Server (AS) is the authoritative server for US. It has two duties. First is to handle the registration requests to pair hostnames to IP, second is to be able to respond to DNS queries from clients.

   i. **Registration**: Accept a UDP connection on port 53533 and register Type A DNS record into the database. Hint: You need to store the value in somewhere persistent, for example in a file so that you can later respond to DNS queries.
   ii. **DNS Query**: Respond to DNS Query on port 53533. Query should include: (Name, Type). If the message conforms to this, server will return the IP address in a DNS message of form: (Name, Value, Type, TTL), retrieved from the file (note that AS can distinguish registration requests from DNS queries by simply looking at the

fields provided). Here is a sample request and response:

Request:

TYPE=A
NAME=fibonacci.com

Response:

TYPE=A
NAME=fibonacci.com VALUE=IP_ADDRESS TTL=10

Problem 2 will be graded based on the following. Please follow the instructions carefully for including the port numbers and message formats. Any deviation from the specification above will result losing points in the assignment:

a. US responds to requests in path /fibonacci as specified above.
b. FS accepts a registration request in /register as specified above.
c. FS responds to GET requests in /fibonacci.
d. AS performs a registration requests as specified above.
e. AS provides DNS record for a given query as specified above.

Please submit the following:

a. Create **dns_app** folder in GitHub and create three separate folders within it: "**US**", "**FS**" and "**AS**".
b. For each folder provide **Dockerfile** and all the files that Dockerfile needs in order to run the specific server.
c. Finally make the zip of "**dns_app**" folder and upload to NYU Brightspace.
d. Your final commit date to the folder in Github will be used to grade the submission.

Extra Credit: Run your application in a cloud K8s cluster (you can pick a cloud platform of your choice). Provide a single file (containing your deployment and service) that can be used to deploy your whole application with: "kubectl apply -f deploy_dns.yml". Be sure to expose your ports with nodePort. Name the single file as "**deploy_dns.yml"**. Also note that K8s does not allow nodePort except for the range of 30000- 32767. Therefore, make sure that in your .yaml file, specify AS nodePort number as **30001**, FS port number as **30002** and US port number as **30003**.

You can take a look at this link to learn more about how to create this deployment file:

https://kubernetes.io/docs/concepts/workloads/controllers/deployment/#creating-a- deployment

Tips:

To build an image from Dockerfile, use the following command (change the tag accordingly and notice the . at the end which specifies the context of the build):

```
docker build -t bulutmf/fs:latest .
```

To get servers running within Docker containers communicate with each other, you can create a Docker network with the following command:

```
docker network create N_NAME
```

Once created run your containers by specifying the network name with the following command (change the parameters accordingly):

```
docker run --network N_NAME --name C_NAME -p 53533:53533/udp -it bulutmf/as:latest
```

Containers that are running within the same network should be able to communicate with each other. You can learn the IP address of your container by inspecting the network that you created with the following command:

```
docker inspect N_NAME
```

VI. <u>Deliverables</u>

1. Electronic:

   Your lab assignment files must be submitted via NYU Brightspace.

   For part V.1, your lab assignment file must be submitted via NYU Brightspace. Name the file "**DCN-firstname_lastname_lab_#.docx**" (e.g., "DCN-john_doe_lab_3.docx"). The file must be created and sent by the beginning of class. After the class period, the homework is late. The Brightspace clock is the official clock.

   For part V.2, submit all your code via NYU Brightspace (as a .zip file). Name the file "**DCN-firstname_lastname_lab_#.zip**" (e.g., "DCN-john_doe_lab_3.zip"). The file must be created and sent by the beginning of class. After the class period, the homework is late. The email clock is the official clock.

   Also make sure to commit your programming question to Github. We will use the last commit time for grading.

   - In Github create a folder for the programming exercise and name it "**dns_app**". Make sure to upload your code in there.

2. Cover page and other formatting requirements:

   The cover page supplied on the next page must be the first page of your lab assignment file.

   Fill in the blank area for each field.

   <u>NOTE</u>:

   The sequence of the electronic submission is:

   1. **Cover sheet**
   2. **Lab Assignment Answer Sheet(s)**

3. Grading guidelines:

   Assignment Layout (15%)
   - Lab Assignment is neatly assembled on 8 1/2 by 11 layout.
   - Cover page with your name (last name first followed by a comma then first name), date, NYU ID, and course section number with a signed statement of independent effort is included.
   - File name is correct.

   Answers to Individual Questions (85%):
   - Answers to all questions are complete and correct.
   - Assumptions provided as required.

   (100 points total, all questions weighted equally)

VII. Sample Cover Sheet

Name _____ Date: _____
(last name, first name)

NYU ID: _____

Section: _____

Lab #3 – Application Layer

Total in points (100 points total): _____

Professor's Comments:

Affirmation of my Independent Effort: _____
(Sign here)